

2110531 - Data Science and Data Engineering Tools

Data Processing with Spark

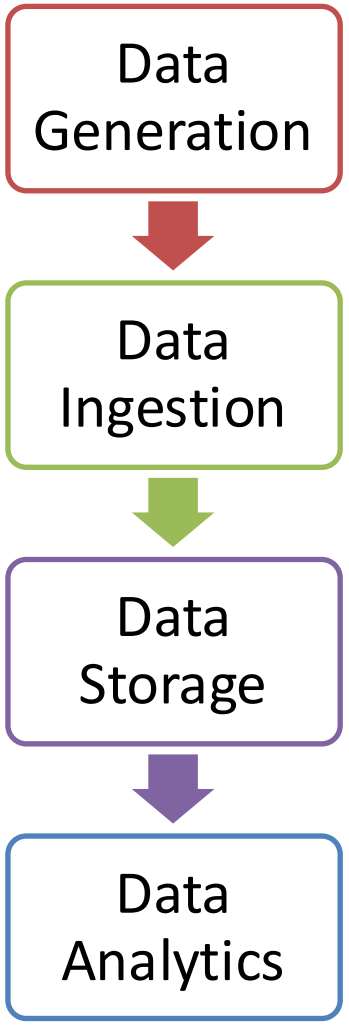
Asst. Prof. Natawut Nupairoj, Ph.D.

Department of Computer Engineering, Chulalongkorn University

natawut.n@chula.ac.th

CHULA ENGINEERING
Foundation toward Innovation

Anatomy of Data Pipeline



- Various data sources
- Various formats
- Real time and batch

- Transactional Databases
- Mobile Applications
- IOT / Sensors

- Data extraction
- Data transportation
- Data pre-processing

- Domain-specific protocol
- Middleware: Kafka, MQTT
- ETL tools

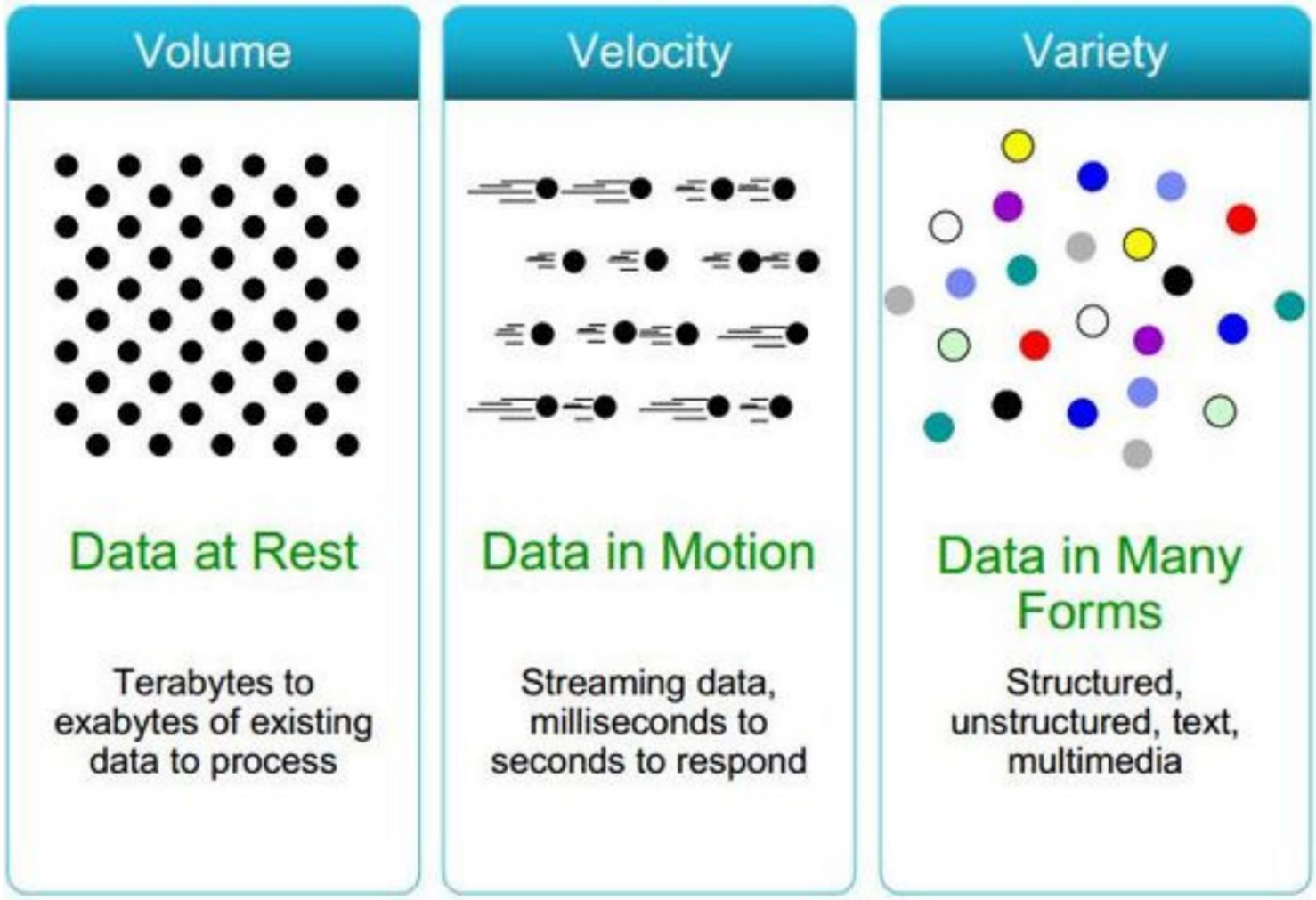
- Big data
- Relational vs NoSQL
- Data Lakehouse
- Query engine

- HDFS
- MongoDB, Cassandra, Redis
- Iceberg, Hudi, Ducklake
- Arrow, Presto, Drill, Trino

- Data processing
- Machine learning / deep learning
- Data visualization

- Pandas, Polars, Dask, Spark, Flink
- ML algorithms
- Power BI, Tableau, Metabase

Data Characteristics - 3V



Source: IBM

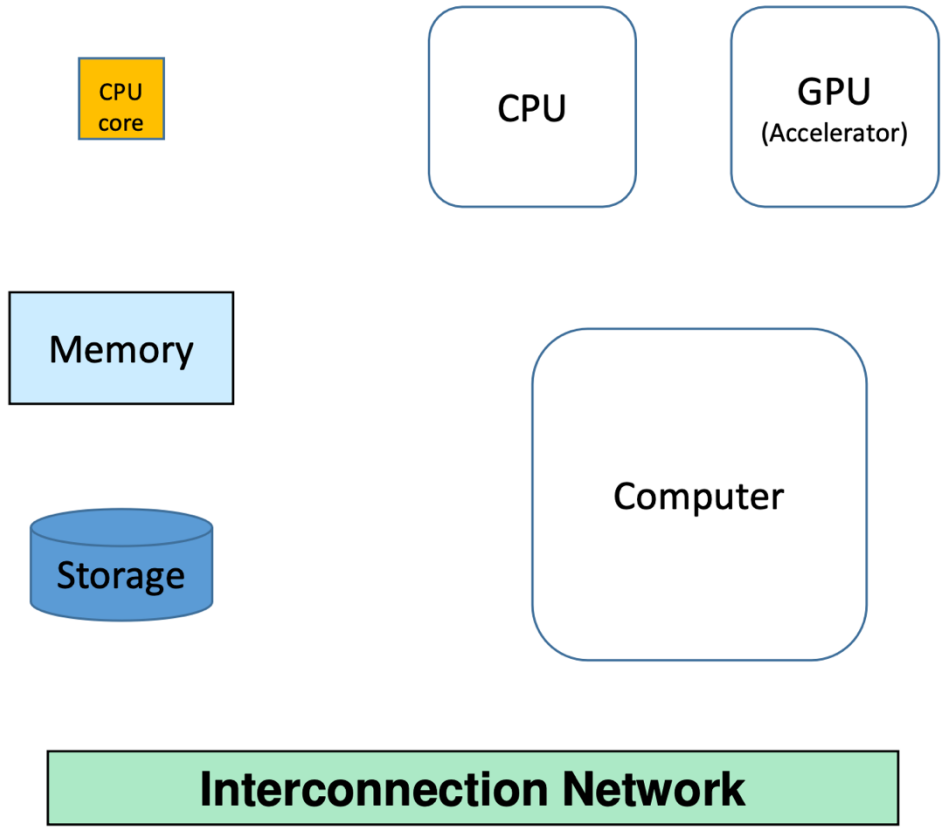
Needs to Understand Processing Architecture

- 3Vs put lots of constraints in processing architecture
  - How we can process large volume of data
  - How we can process stream data in real-time
  - How we can process unstructured e.g. multimedia data
- To solve these problems effectively, we will need to understand and effectively utilize the underlying architecture

Basic Building Blocks

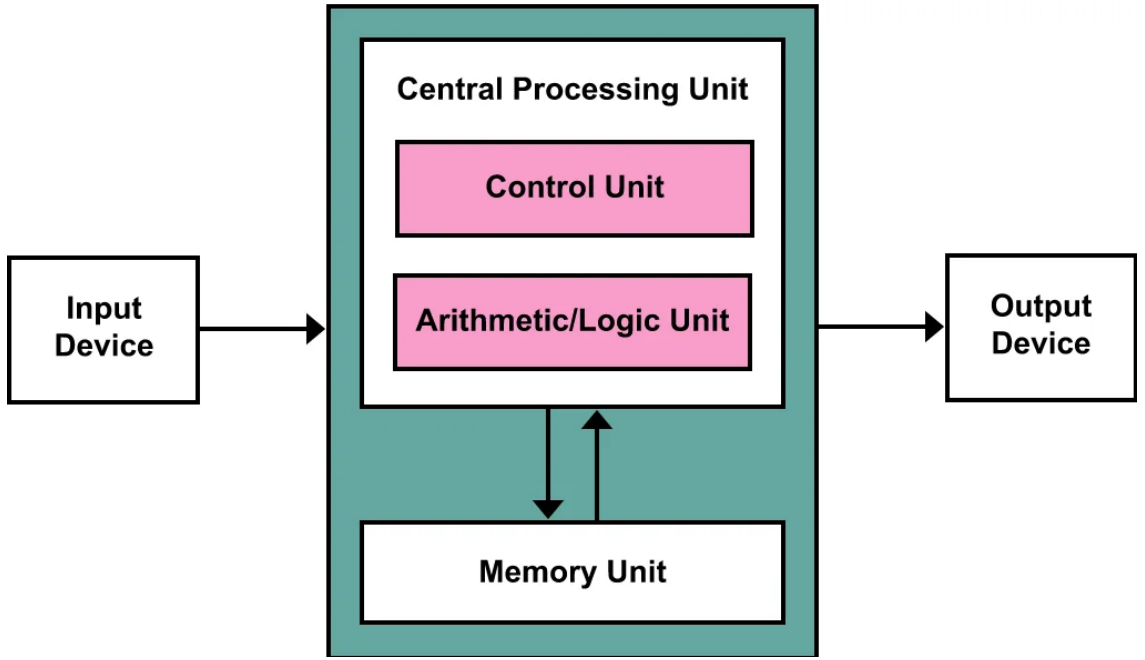


Building Block of a Computer System

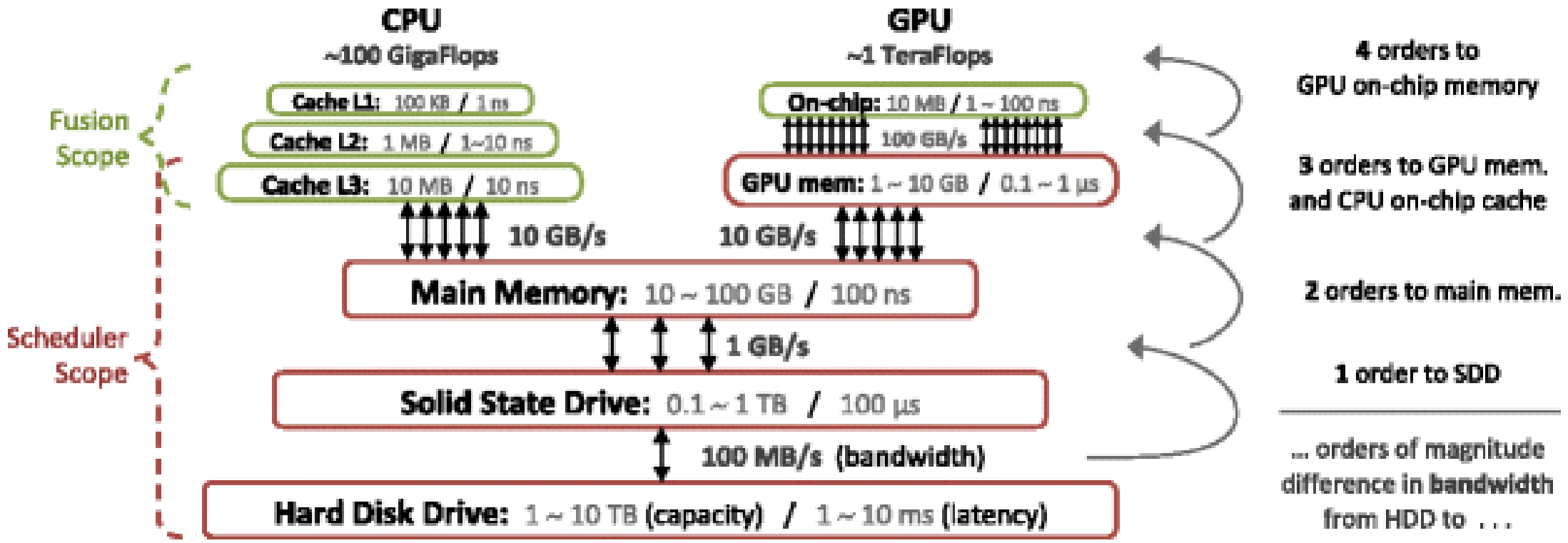


Von Neumann Architecture

- Also known as "stored-program computer”
- both program instructions and data are kept in (same) memory
- Have been the foundation of all CPUs
- Fetching an instruction and performing a data operation cannot occur at the same time because they share a common bus
  “Von Neumann Bottleneck”

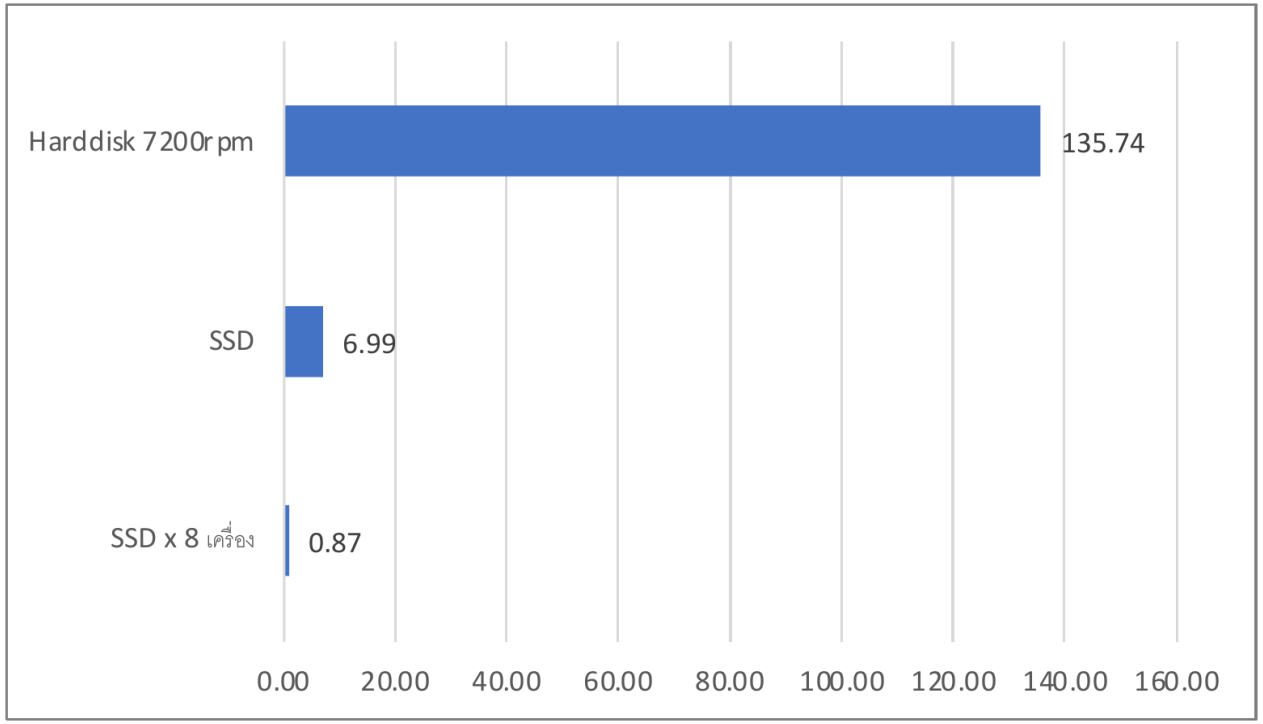


Memory Hierarchy and Bandwidth



https://www.researchgate.net/publication/321150603_A_compiler_approach_to_map_algebra_automatic_parallelization_locality_optimization_and_GPU_acceleration_of_raster_spatial_analysis

Limitations of Disk Data Transfer Bandwidth



128 MB/s

2.5 GB/s

20 GB/s

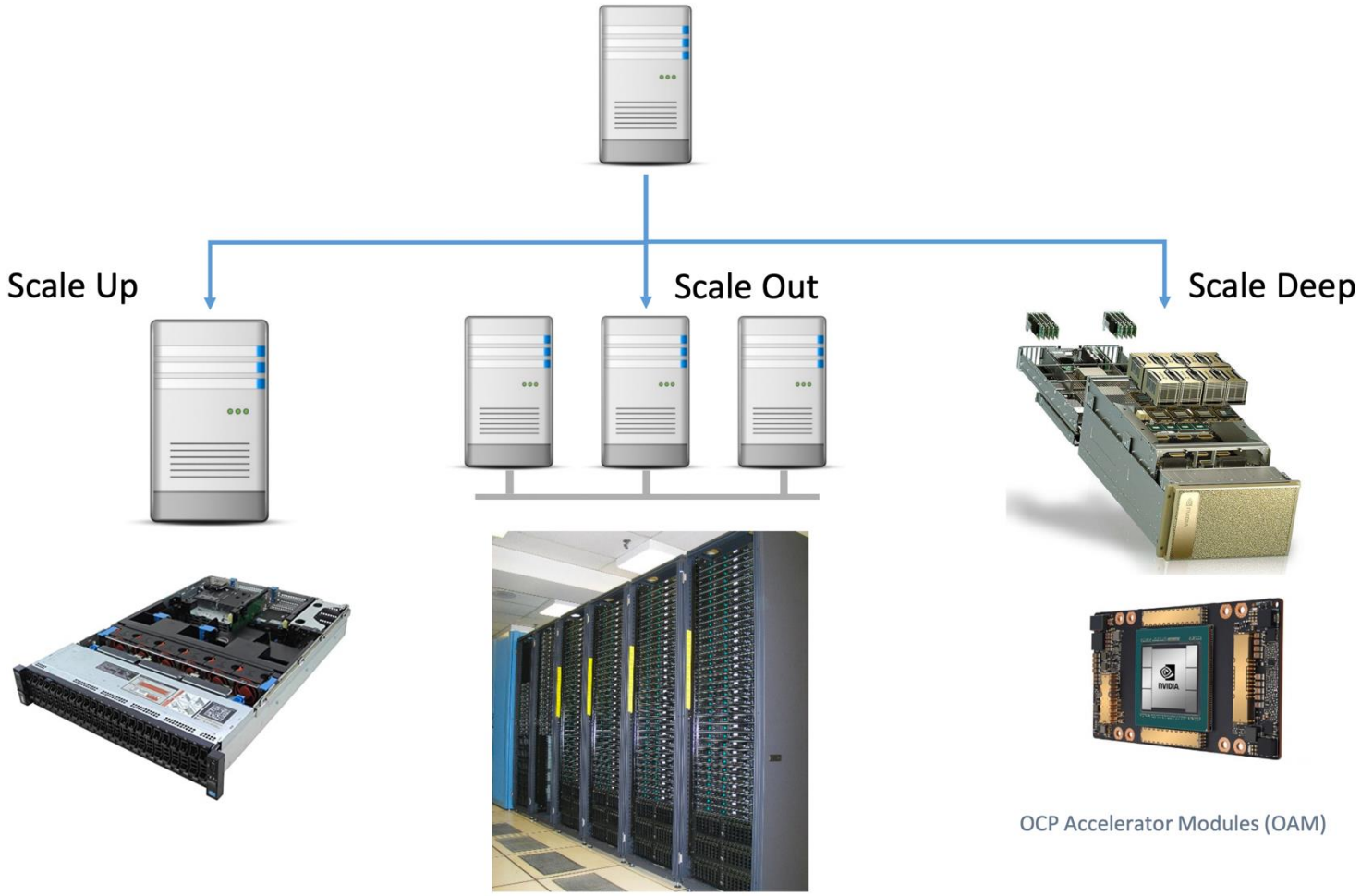
Theoretical time (in minutes) to read 1-TB data from disks

Apple M5 Max Memory Bandwidth = 614 GB/s

Time to transfer 1-TB data = 0.028 minutes

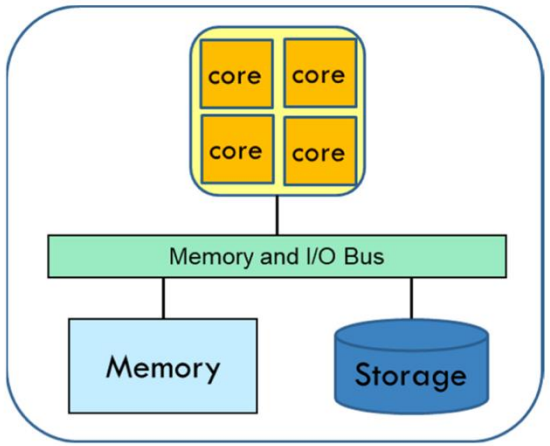
CHULA ENGINEERING
Foundation toward Innovation

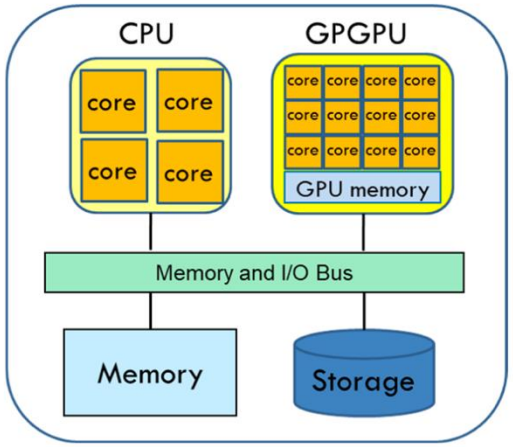
Scale Up / Scale Out / Scale Deep



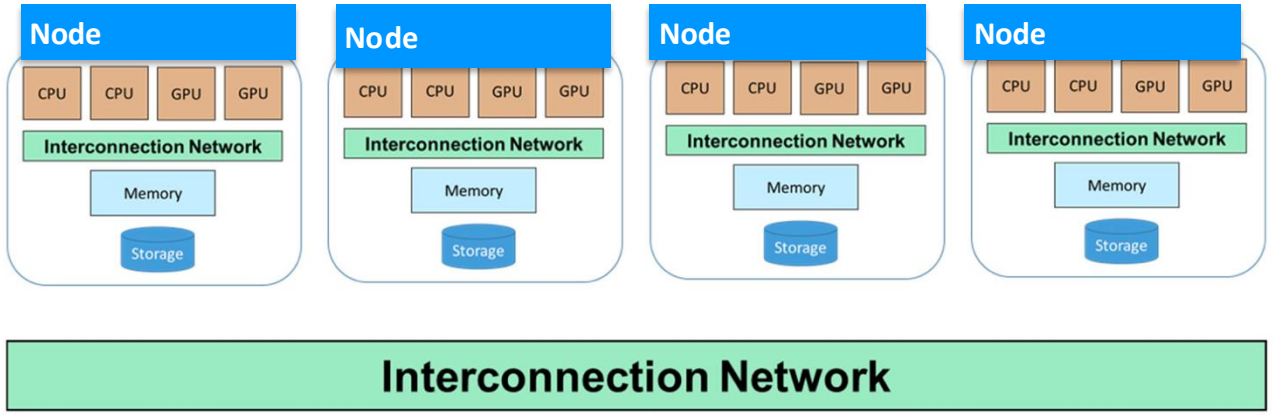
Cluster Computing







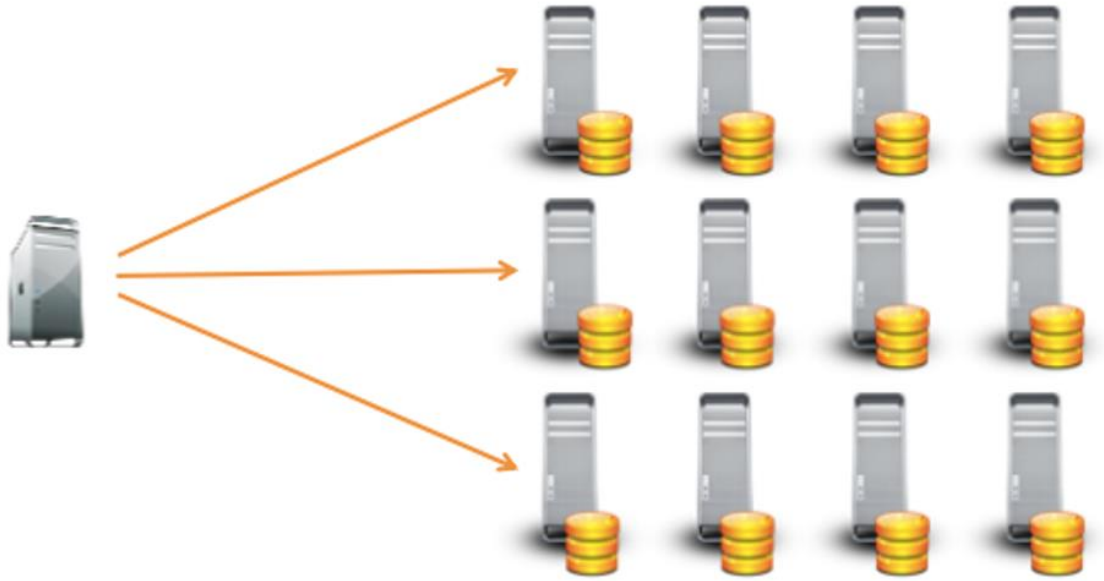




Parallelism

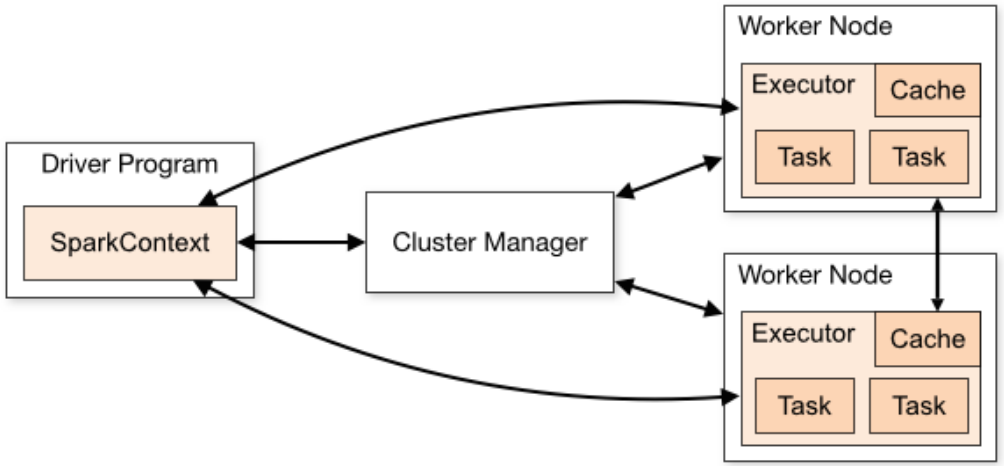
- Big Data relies on the concept of Parallelism on Cluster Computing
- Parallelism is the potential to divide a task into sub-tasks and perform them in parallel
- Parallel computing involves how to exploit parallelism in software with parallelism in hardware (parallel architecture)

Principles of MapReduce



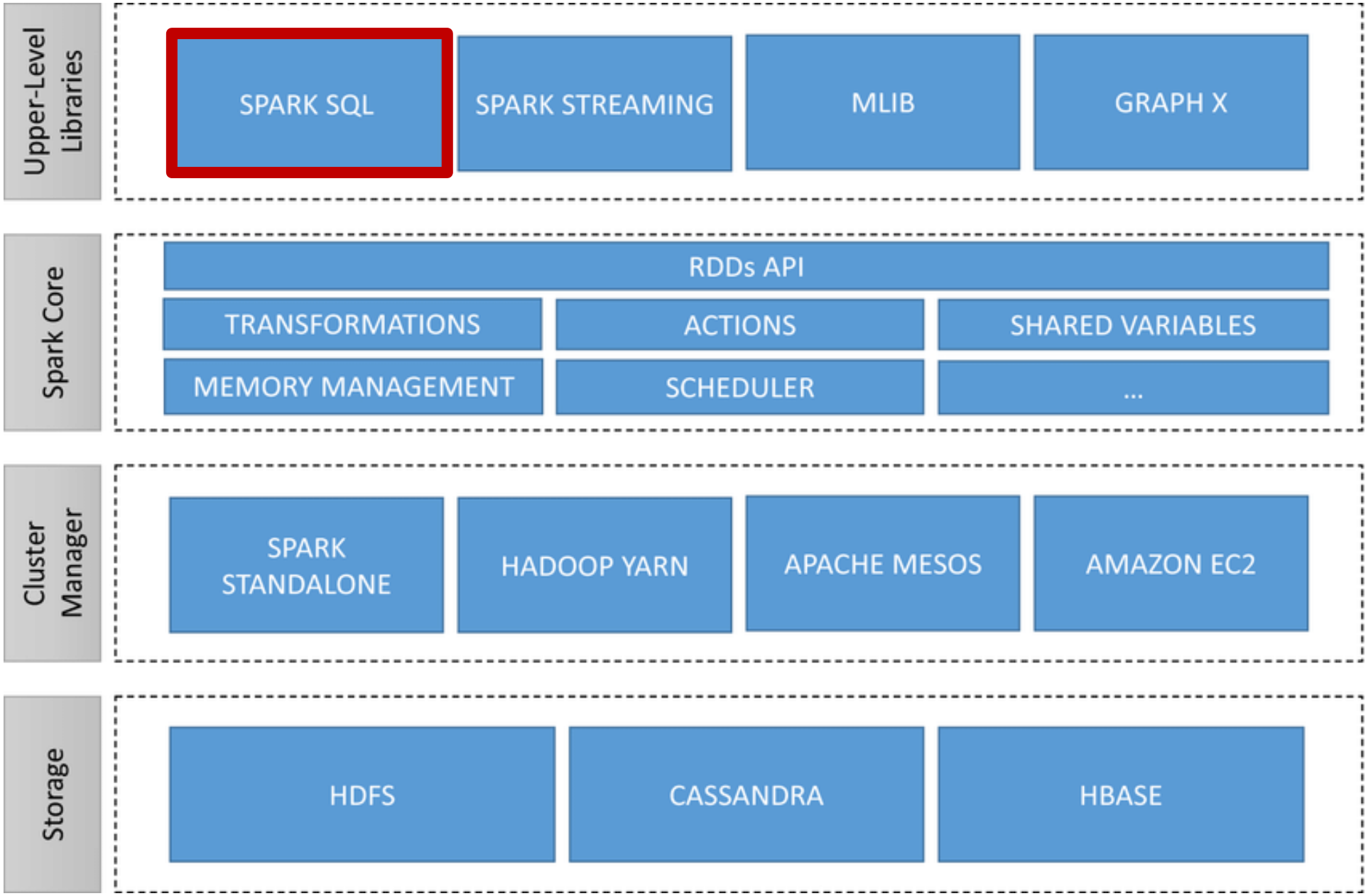
1. Data is distributed across machine
2. MAP – all nodes process data
3. REDUCE – gather results back

Spark basics



Apache Spark

- Spark = In-Memory Data Processing
  - Extend MapReduce model to support interactive queries and stream processing
  - Keep data in memory as long as possible
  - Spark supports interactive queries with Spark Shell (Scala, Python, and R)
- Data Scientist Friendly
  - Low latency (interactive) queries on historical data
  - Low latency queries on live data (streaming)
  - Compatibility with popular systems (Hadoop)



N. Fikri, et al. "An adaptive and real-time based architecture for financial data integration", Journal of Big Data, Nov. 2019

Spark and Big Data



One machine

- pandas loads all data into one computer's memory
- Data bigger than RAM: slow or impossible
- One CPU core does most of the work



Spark: scale out

- Data split into partitions across many cores or machines
- Same code on a laptop or a 100-node cluster
- Batch, SQL, streaming and ML in one engine

Use Spark when…	Something simpler is fine when…
Data is larger than one machine's memory	Data fits easily in memory (MetroRide: 269 rows)
Jobs must scale out on a cluster	You run quick analysis on a laptop: pandas, Polars, DuckDB

One Driver, Many Executors



Driver

- Runs your PySpark program
- Builds and optimises the plan
- Splits the plan into tasks
- Receives action results

tasks →



Executor 1

task · partition 1

task · partition 2



Executor 2

task · partition 3

task · partition 4



Executor 3

task · partition 5

task · partition 6



Cluster manager (YARN, Kubernetes, standalone) hands out executors
master("local[*]") = driver and executors in one process, one task slot per CPU core.

CHULA ENGINEERING
Foundation toward Innovation

Running pyspark locally

- Spark runs on the JVM; check with java –version (should be > 17)
- Install pyspark

```
pip install pyspark
```

- How to run pyspark
  - Running CLI (this will create SparkSession automatically)

```
pyspark
```

  - Running via Jupyter Lab (you will have to create SparkSession manually)

```
SparkSession.builder.master('local').appName('app1').getOrCreate()
```

PySpark: Python Driving a JVM Engine




```
from pyspark.sql import SparkSession

spark = (SparkSession.builder
         .master("local[*]")
         .appName("spark-intro")
         .getOrCreate())
print(spark.version)
print(spark.sparkContext.defaultParallelism)
```

- Python sends instructions; the work runs in the JVM
- SparkSession is the entry point to everything
- getOrCreate() reuses a running session
- Spark UI at localhost:4040 shows jobs, stages, tasks

output

```
4.2.0
2       # task slots = CPU cores here
```

spark.stop() ends the session and frees its memory.

CHULA ENGINEERING
Foundation toward Innovation

DataFrame: Rows, Columns, Schema, Partitions

```
trips = spark.createDataFrame(
    [("T1", "Siam", 148.0),
     ("T2", "Asok", None),
     ("T3", "Ari", 95.5)],
    "trip_id string, zone string, fare double")
trips.printSchema(); trips.show()
trips.rdd.getNumPartitions()
```



Schema — Column names and types, known before any data is read



Partitions — Chunks of rows; each task processes one



Immutable — Operations return a new DataFrame

output

```
root
 |-- trip_id: string (nullable = true)
 |-- zone: string (nullable = true)
 |-- fare: double (nullable = true)

+-------+----+-----+
|trip_id|zone| fare|
+-------+----+-----+
|     T1|Siam|148.0|
|     T2|Asok| NULL|
|     T3| Ari| 95.5|
+-------+----+-----+
2
```

NULL is Spark's missing value (Python None).

Spark Types and Python Types

Python value	Inferred Spark type	Note
`"Siam"  (str)`	`string`	
`3  (int)`	`long`	64-bit; IntegerType is 32-bit
`1.5  (float)`	`double`	not exact for money
`Decimal("148.00")`	`decimal(38,18)`	declare decimal(10,2) instead
`datetime(…)`	`timestamp`	an instant in time
`date(…)`	`date`	
`True  (bool)`	`boolean`	
`None`	`NULL`	any type

Declare the schema explicitly

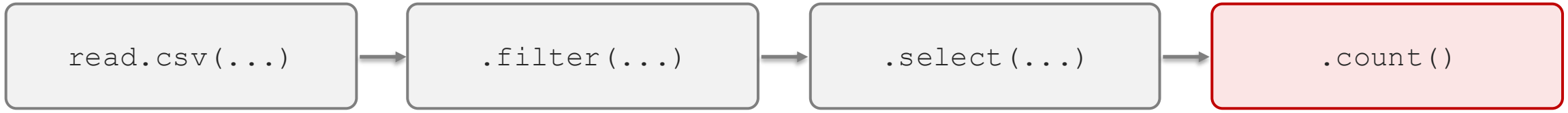
```
from pyspark.sql import types as T

schema = T.StructType([
  T.StructField("trip_id",
                T.StringType()),
  T.StructField("fare_thb",
                T.DecimalType(10, 2)),
])
```



Explicit schemas make pipelines predictable: no guessing, no surprises.

Transformations Are Lazy; Actions Run Jobs



transformations: build a plan, nothing runs yet

action: runs a job

```
completed = raw.filter(F.col("status") == "completed")
fares = completed.select("trip_id", "fare_thb")
# ... nothing has been read yet ...
fares.count()           # action: Spark runs a job
```

output

```
124
```

Transformations (lazy)	Actions (run a job)
select, withColumn, filter	show, count
join, groupBy, orderBy	first, collect
union, distinct, drop	toPandas, toArrow, write

Plans and Caching

```
fares.explain()
```

output

```
== Physical Plan ==
*(1) Project [trip_id#25, fare_thb#34]
+- *(1) Filter (isnotnull(status#33) AND (status#33 = completed))
   +- FileScan csv [trip_id#25,status#33,fare_thb#34] ... PushedFilters:
[IsNotNull(status), EqualTo(status,completed)],
      ReadSchema: struct<trip_id:string,status:string,fare_thb:string>
```

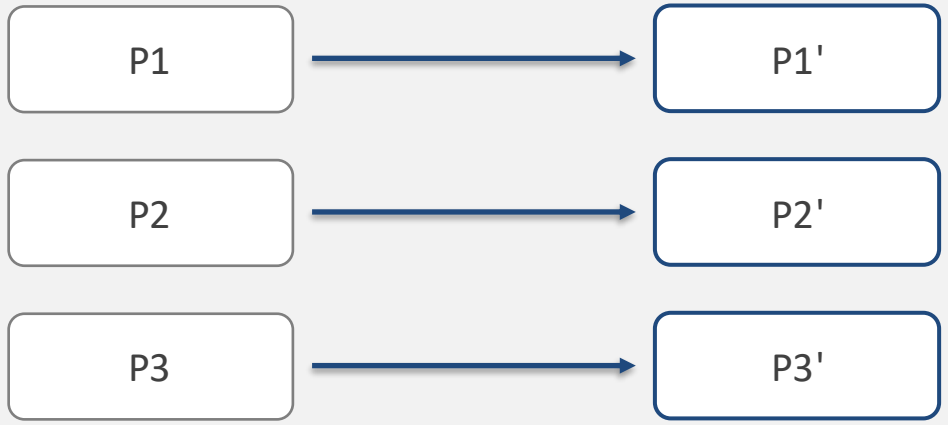


Catalyst optimiser: pushes the filter into the file scan and reads only the 3 columns it needs.

```
fares.cache()     # keep after first use
fares.count()     # job 1: reads the CSV
fares.show(5)     # reuses the cache
fares.unpersist()
```

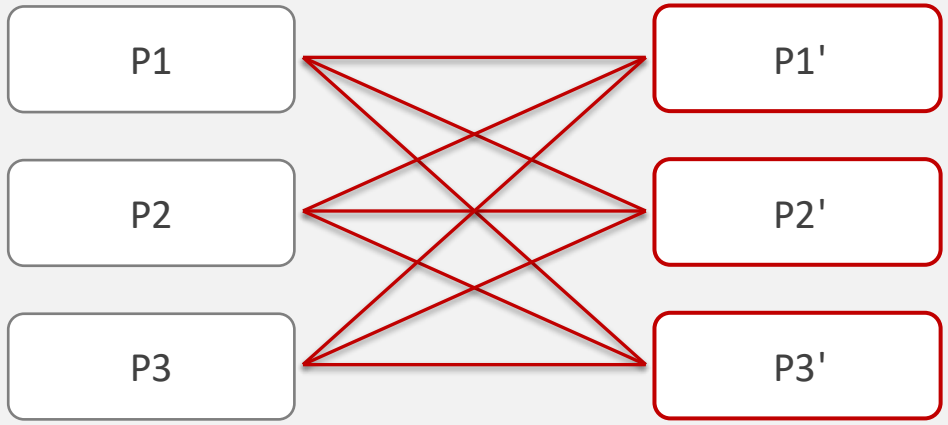
Narrow vs. Wide Transformations

Narrow: no data movement



`select · filter · withColumn · cast`

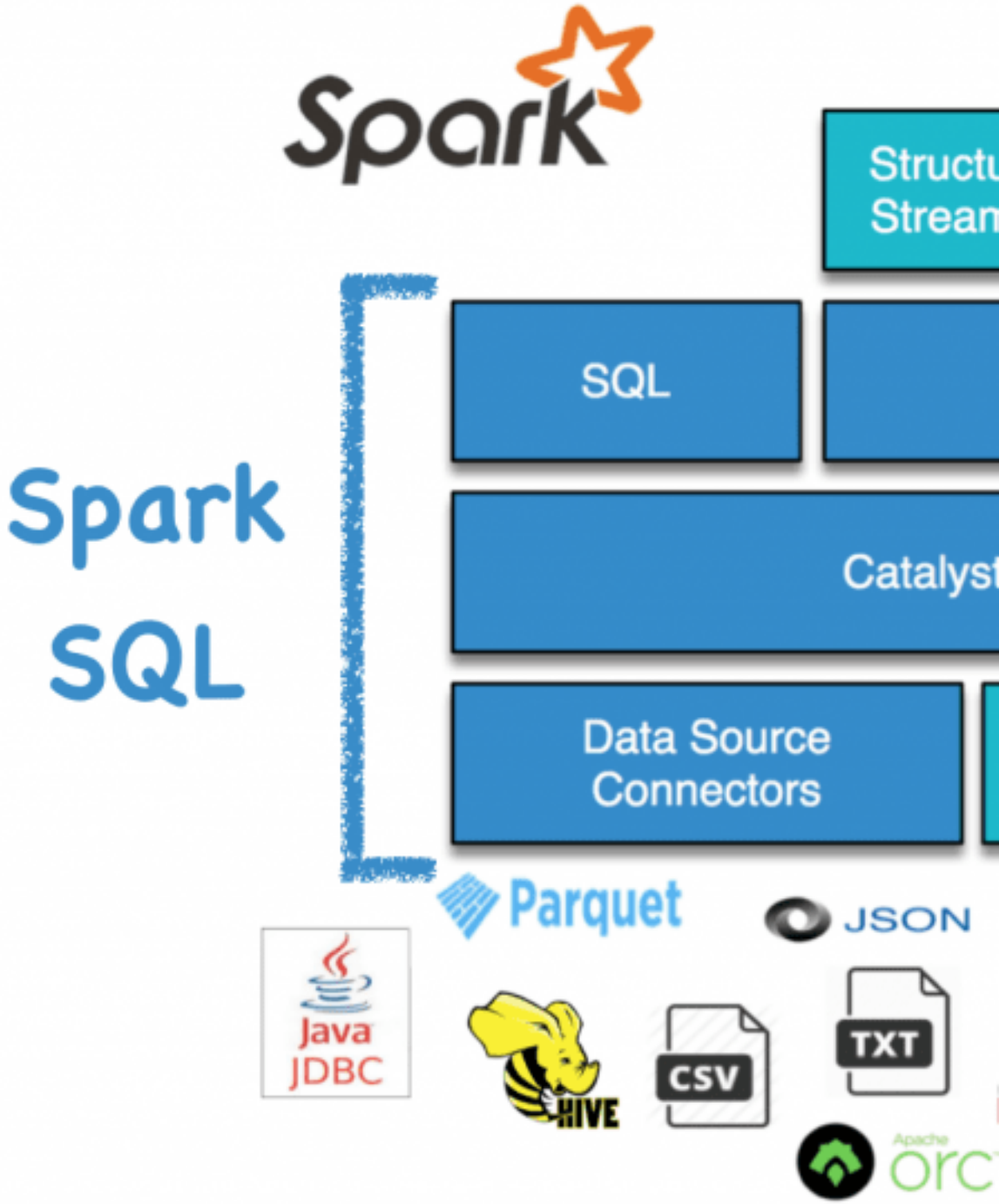
Wide: shuffle between partitions



`join·groupBy·window·distinct·orderBy`

```
spark.conf.set("spark.sql.shuffle.partitions", "4")    # default 200: too many for
small data
```

Working with DataFrames



Reading CSV Files

```
COLS = ["trip_id", "driver_id", "service_zone", ...]
schema = T.StructType(
    [T.StructField(c, T.StringType()) for c in COLS])

raw = (spark.read
       .option("header", True)
       .option("enforceSchema", False)
       .schema(schema)
       .csv("trips_2026-04.csv"))
```

Option	Meaning
`header`	first line = column names
`schema(...)`	use these names and types
`inferSchema`	guess types (extra pass)
`enforceSchema=False`	check header vs schema
`blank cell`	read as NULL

output

```
raw.filter("status = 'cancelled'")
   .select("trip_id", "picked_up_at", "fare_thb").show(3)
|        trip_id|picked_up_at|fare_thb|
|BKK-20260405-M1|        NULL|    NULL|
|BKK-20260406-E3|        NULL|    NULL|
```

output

```
inferSchema=True guesses:
 |-- requested_at: timestamp
 |-- fare_thb: double
```

Other Formats and Whole Files

Format	Read with	Schema comes from
CSV	`spark.read.csv(path, header=True)`	you, or inferSchema
Parquet	`spark.read.parquet(path)`	the file itself
JSON	`spark.read.json(path)`	inferred from the data
Whole files	`spark.read.format("binaryFile")`	path, length, content…

```
sha = (spark.read.format("binaryFile").load("trips_2026-04.csv")
         .select(F.sha2("content", 256).alias("sha256"))
         .first()["sha256"])
```

output

```
ee6286bcdafff7d1…
```

Same bytes always give the same hash, so a file that was already loaded can be detected.

CHULA ENGINEERING
Foundation toward Innovation

Column Expressions

```
from pyspark.sql import functions as F

t = raw
  .select("trip_id",
          F.trim("service_zone").alias("zone"),
          F.lower("status").alias("status"), "fare_thb")
  .withColumn("fare", F.col("fare_thb").cast("double"))
  .withColumn("price_band",
      F.when(F.col("fare") >= 170, "high")
       .when(F.col("fare") >= 140, "mid")
       .otherwise("low"))
  .filter(F.col("zone") != "Silom")
  .drop("fare_thb")
```

output

```
|        trip_id|zone|   status| fare|price_band|
|BKK-20260401-M1|Asok|completed|148.0|       mid|
```

Method / function	Does
`select(...)`	choose or compute columns
`withColumn(name, expr)`	add or replace one column
`filter / where`	keep rows where True
`drop(...)`	remove columns
`F.col, F.lit`	column, constant
`F.when(...).otherwise(...)`	if / else
`alias(name)`	rename a result

Logical Operators: And - Or - Not

Right: one pair of parentheses per comparison

```
raw.filter((F.col("status") == "completed")
           & (F.col("service_zone") == "Ari")).count()
raw.filter(~F.col("service_zone").isin("Siam", "Asok")).count()
```

output

```
31
67
```

and → & **or → |** **not → ~** also: isin(...), isNull(), isNotNull(), between(a, b)

Wrong: missing parentheses

```
raw.filter(F.col("status") == "completed" & F.col("service_zone") == "Ari")
```

error

```
PySparkValueError: [CANNOT_CONVERT_COLUMN_INTO_BOOL] Cannot convert column into bool: please use '&' for 'and',
'|' for 'or', '~' for 'not' when building DataFrame boolean expressions.
```

NULL Means "Unknown"

```
df = spark.createDataFrame([("T1", 148.0), ("T2", None)],
                           "trip_id string, fare double")
df.select("trip_id", "fare",
          (F.col("fare") > 100).alias("fare_gt_100")).show()
df.filter(F.col("fare") > 100).count()
df.filter(~(F.col("fare") > 100)).count()
df.filter(F.col("fare").isNull()).count()
```

output

```
|trip_id| fare|fare_gt_100|
|     T1|148.0|       true|
|     T2| NULL|       NULL|

1    # fare > 100
0    # NOT (fare > 100): T2 is in neither!
1    # fare IS NULL
```

Expression	Result
NULL > 100	NULL
TRUE AND NULL	NULL
FALSE AND NULL	FALSE
TRUE OR NULL	TRUE
NOT NULL	NULL



filter keeps TRUE only. Rows with NULL conditions vanish, which is how invalid rows are dropped in the tutorial.

Parsing and Casting: cast vs. try_cast

```
TS = "yyyy-MM-dd'T'HH:mm:ssXXX"
bad.select(
    F.try_to_timestamp("ts", F.lit(TS)).alias("ts"),
    F.col("fare").try_cast("decimal(10,2)").alias("fare")).show()
```

output

```
|                 ts|  fare|
|2026-04-01 08:03:00|148.00|
|               NULL|  NULL|    # 'not a time', 'abc'
```

```
bad.select(F.col("fare").cast("decimal(10,2)")).show()
```

error

```
[CAST_INVALID_INPUT] The value 'abc' ... cannot be cast to "DECIMAL(10,2)"
```

Pattern	Matches
`yyyy-MM-dd`	2026-04-01
`'T'`	literal T
`HH:mm:ss`	08:03:00 (24-hour)
`XXX`	+07:00 offset



ANSI mode is on in Spark 4: cast and to_timestamp fail on bad input. Use try_cast and try_to_timestamp.

Time Zones

```
one = (spark.createDataFrame(
           [("2026-04-01T06:30:00+07:00",)], "raw string")
         .select(F.to_timestamp("raw", TS).alias("ts")))

for tz in ["UTC", "Asia/Bangkok"]:
    spark.conf.set("spark.sql.session.timeZone", tz)
    one.select("ts", F.to_date("ts").alias("date"),
               F.hour("ts").alias("hour")).show()
```

output

```
UTC
|                 ts|      date|hour|
|2026-03-31 23:30:00|2026-03-31|  23|
Asia/Bangkok
|                 ts|      date|hour|
|2026-04-01 06:30:00|2026-04-01|   6|
```



Same instant

Stored once; only the display and date/hour change.



Set it once

spark.sql.session.timeZone in the builder.



Derive safely

to_date(), hour(), date_format() use it.

Joins

```
trips   # (T1, D1) (T2, D2) (T3, D9)
drivers # (D1, Siam) (D2, Ari)
trips.join(drivers, "driver_id", how)
```

how	Keeps
inner	rows with a match on both sides
left	all left rows; NULLs where no match
right / full	all right rows / all rows
left_semi	left rows that have a match (no right columns)
left_anti	left rows that have NO match

trip_id	drive_id
T1	D1
T2	D2
T3	D3

drive_id	home_zone
D1	Siam
D2	Ari

how="inner"

```
|driver_id|trip_id|home_zone|
|       D1|     T1|     Siam|
|       D2|     T2|      Ari|
```

how="left"

```
|driver_id|trip_id|home_zone|
|       D1|     T1|     Siam|
|       D9|     T3|     NULL|
|       D2|     T2|      Ari|
```

how="left_anti"

```
|driver_id|trip_id|
|       D9|     T3|
```

CHULA ENGINEERING
Foundation toward Innovation

Aggregations

- agg() (aggregate) is used to calculate summary statistics such as sum, average, count, minimum, and maximum from a DataFrame
- It is commonly used with groupBy() to aggregate records by one or more columns.

```
raw.groupBy("service_zone")
    .agg(F.count("*").alias("trips"))
    .show()
```

output

```
|service_zone|trips|
|         Ari|   33|
|        Asok|   32|
|        Siam|   33|
|       Silom|   34|
```

Aggregations

```
raw.groupBy("service_zone")
    .agg(F.count("*").alias("trips"),
         F.count_if(F.col("status") == "completed")
          .alias("completed"),
         F.sum(F.col("fare_thb").cast("decimal(10,2)"))
          .alias("fare_thb"),
         F.countDistinct("driver_id").alias("drivers"))
    .orderBy("service_zone").show()
```

output

```
|service_zone|trips|completed|fare_thb|drivers|
|         Ari|   33|       31| 5676.00|     12|
|        Asok|   32|       30| 5325.00|     12|
|        Siam|   33|       32| 5560.00|     12|
|       Silom|   34|       31| 6014.00|     12|
```

Function	Returns
`F.count("*")`	number of rows
`F.count_if(cond)`	rows where cond is True
`F.countDistinct(c)`	distinct values
`F.sum / avg / min / max`	the usual
`F.first(c)`	first value seen

Aggregates skip NULLs, except count(""), which counts rows.*

Window Functions

- A Window function in PySpark performs calculations across a group of related rows without collapsing those rows into a single result
- This is the key difference from `groupBy().agg()`:
  - `groupBy().agg()` reduces multiple rows into summary rows
  - Window functions preserve the original rows while adding calculated columns
- Window functions are particularly useful for ranking, running totals, moving averages, comparing records, and time-series analysis

CHULA ENGINEERING
Foundation toward Innovation

Window Functions

```
from pyspark.sql import Window
from pyspark.sql import functions as F

w = Window.partitionBy("product")

result = df.withColumn(
    "total_sales",
    F.sum("sales").over(w)
)
```

product	month	sales	total_sales
A	Jan	100	450
A	Feb	200	450
A	Mar	150	450
B	Jan	300	950
B	Feb	250	950
B	Mar	400	950

product	month	sales
A	Jan	100
A	Feb	200
A	Mar	150
B	Jan	300
B	Feb	250
B	Mar	400

```
from pyspark.sql import functions as F

df.groupBy("product").agg(
    F.sum("sales").alias("total_sales")
)
```

product	total_sales
A	450
B	950

CHULA ENGINEERING
Foundation toward Innovation

Window Functions

```
from pyspark.sql import Window
from pyspark.sql import functions as F

w = Window.partitionBy("trip_id").orderBy(F.desc("version"))

# ver is a dataframe with trip_id, status, and version
ver.withColumn("rn", F.row_number().over(w)).show()
```

output

```
|trip_id|    status|version| rn|
|     T1|cancelled|      2|  1|
|     T1|completed|      1|  2|
|     T2|completed|      1|  1|
```

```
latest = (ver.withColumn("rn", F.row_number().over(w))
             .filter("rn = 1").drop("rn"))  # 1 per trip
```



`partitionBy`

Which rows belong together (one group per trip_id).



`orderBy`

Order inside each group (newest version first).



`row_number()`

Assigns sequential row numbers starting at 1 for each group (groupby trip_id)

Spark Transformations vs. Pandas Operations

PySpark transformation	Wide or Narrow	Comparable Pandas operation
select()	Narrow	df[['col1', 'col2']]
withColumn()	Narrow	df.assign()
drop()	Narrow	df.drop(columns=[...])
filter() / where()	Narrow	df.loc[condition]
distinct()	Wide	df.drop_duplicates()
dropDuplicates()	Wide	df.drop_duplicates(subset=[...])
groupBy().agg()	Wide	df.groupby().agg()
groupBy().count()	Wide	df.groupby().size()
orderBy() / sort()	Wide	df.sort_values()

Spark Transformations vs. Pandas Operations

PySpark transformation	Wide or Narrow	Comparable Pandas operation
join()	Wide (typically)	pd.merge()
union()	Narrow	pd.concat([df1, df2])
intersect()	Wide	pd.merge() + drop_duplicates()
subtract() / exceptAll()	Wide	Anti-join using pd.merge()
sample()	Narrow	df.sample()
limit()	Narrow*	df.head(n)
explode()	Narrow	df.explode()
fillna()	Narrow	df.fillna()
dropna()	Narrow	df.dropna()

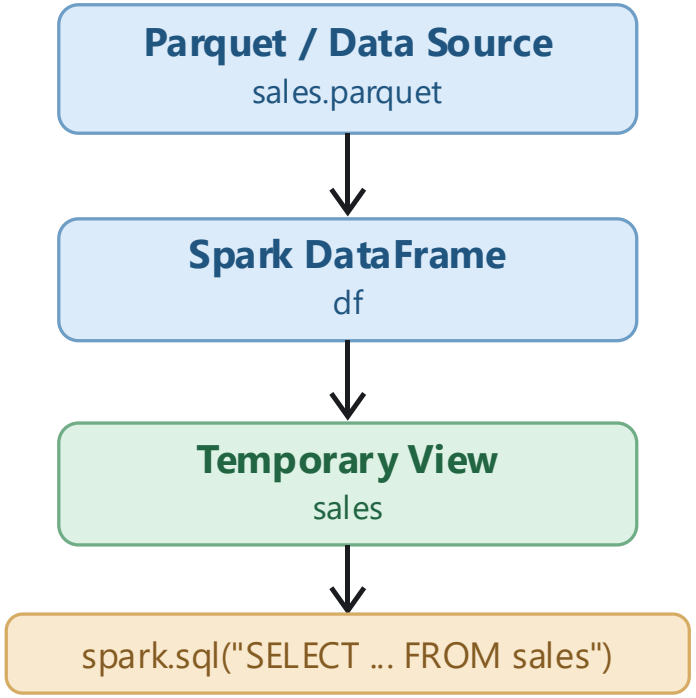
Spark Transformations vs. Pandas Operations

PySpark transformation	Wide or Narrow	Comparable Pandas operation
replace()	Narrow	df.replace()
withColumnRenamed()	Narrow	df.rename(columns={...})
pivot()	Wide	pd.pivot_table()
unpivot() / melt()	Narrow	pd.melt()
map() (RDD)	Narrow	Series.map() / DataFrame.apply()
flatMap() (RDD)	Narrow	explode() / apply()
reduceByKey() (RDD)	Wide	df.groupby().agg()
groupByKey() (RDD)	Wide	df.groupby()
sortByKey() (RDD)	Wide	df.sort_values()

Temporary View

- A Temporary View gives a DataFrame a name so that it can be referenced in SQL
- A Temporary View is a logical view, not a new physical table

```
df = spark.read.parquet("sales.parquet")
df.createOrReplaceTempView("sales")
spark.sql("""
    SELECT product, SUM(amount) AS revenue
    FROM sales
    GROUP BY product
""").show()z
```



Spark SQL and Temporary Views

```
raw.createOrReplaceTempView("trips")
spark.sql("""
  SELECT service_zone, count(*) AS trips FROM trips
  WHERE status = 'cancelled' GROUP BY service_zone
  ORDER BY service_zone""").show()
```

```
(raw.filter(F.col("status") == "cancelled")
    .groupBy("service_zone").agg(F.count("*").alias("trips"))
    .orderBy("service_zone").show())
```

output

```
|service_zone|trips|
|         Ari|    2|
|        Asok|    2|
|        Siam|    1|
|       Silom|    3|

(identical for both)
```



One engine, two languages. A temp view lives only in this SparkSession; it is a name for a DataFrame, not a stored table.

Getting Results Out

Action	Returns	Size
`show(n, truncate=False)`	printed table	first n rows
`show(vertical=True)`	one block per row	first n rows
`first()`	one Row	1 row
`collect()`	list of Row	ALL rows
`toPandas()`	pandas DataFrame	ALL rows
`toArrow()`	pyarrow Table	ALL rows

```
raw.first()["trip_id"]
raw.select("trip_id", "status").limit(2).collect()
```

output

```
'BKK-20260401-M1'
[Row(trip_id='BKK-20260401-M1',
     status='completed'),
 Row(trip_id='BKK-20260401-M2', ...)]
```



collect, toPandas and toArrow pull every row into the driver. limit() or aggregate first on big data.

Arrow: Spark Dataframe and Python Table



Spark DataFrame



Arrow table



pandas · pyducklake · DuckDB

```
spark.conf.set("spark.sql.execution.arrow.pyspark.enabled",
               "true")
tbl  = small.toArrow()                  # pyarrow.Table
pdf  = small.toPandas()                 # pandas
back = spark.createDataFrame(tbl)       # Arrow -> Spark
```

tbl.schema

```
trip_id: string
requested_at: timestamp[us, tz=UTC]
fare_thb: decimal128(10, 2)
```



Type details: timestamps leave Spark in UTC; pandas receives Decimal objects. The tutorial casts Arrow tables to the DuckLake table's exact types before appending.

Writing Data with Spark

```
(raw.select("trip_id", "service_zone", "status")
    .write.mode("overwrite").partitionBy("service_zone")
    .parquet("out/trips_parquet"))
```

files written

```
out/trips_parquet/_SUCCESS
out/trips_parquet/service_zone=Ari/part-00000-….snappy.parquet
out/trips_parquet/service_zone=Asok/part-00000-….snappy.parquet
out/trips_parquet/service_zone=Siam/part-00000-….snappy.parquet
out/trips_parquet/service_zone=Silom/part-00000-….snappy.parquet
```

error

```
write without mode on an existing path:
[PATH_ALREADY_EXISTS] ... Set mode as "overwrite" ...
```

mode	If the path exists
`error (default)`	fail
`append`	add new files
`overwrite`	replace everything
`ignore`	do nothing



A folder is not a table: no transactions, no snapshots. A crash mid-write leaves partial files.

From Files to Lakehouse Tables



Data files

Parquet files written by Spark executors.



Table format

Metadata lists which files form each snapshot; one atomic commit per write.



Catalog in Spark

spark.sql.catalog.<name> plugs Iceberg or Delta into Spark; then df.writeTo("lake.silver.trips").append().



In our tutorial: Spark does all processing; pyducklake creates the DuckLake tables and commits each Spark DataFrame (via Arrow) as a snapshot. Same idea, different connector.

Pipeline Patterns in PySpark



File guard

Skip a file that was already loaded

```
already = bronze.filter(F.col("file_sha256") == sha)
if already.count() > 0: return    # skip the file
```



Row guard

Keep the first copy per key, then only new keys

```
w = Window.partitionBy("trip_id").orderBy(...)
new = batch.join(old, "trip_id", "left_anti")
```



Incremental + idempotent

Process only rows not in the target yet

```
todo = bronze.join(silver.select("trip_id"),
                   "trip_id", "left_anti")
```



Validation

Bad values become NULL, then fail the filter

```
F.col("fare").try_cast("decimal(10,2)")
.filter(rules_all & (ok_completed | ok_cancelled))
```

Reading PySpark Errors

Error class	Typical cause	Fix
`UNRESOLVED_COLUMN`	Column name typo: raw.select("fare")	Use a suggested name (fare_thb)
`CANNOT_CONVERT_COLUMN_INTO_BOOL`	and / or / not, or missing parentheses	Use & | ~ with parentheses
`CAST_INVALID_INPUT`	cast() on a malformed value (ANSI mode)	try_cast, try_to_timestamp
`PATH_ALREADY_EXISTS`	write to an existing path	mode("overwrite") or "append"

error

```
AnalysisException: [UNRESOLVED_COLUMN.WITH_SUGGESTION] A column, variable, or function parameter with name `fare`
cannot be resolved. Did you mean one of the following? [`fare_thb`, `status`, `trip_id`, ...]
```



Read: your last Python line → the [ERROR_CLASS] → the suggestion. Quiet the logs with spark.sparkContext.setLogLevel("ERROR").